What's pledged stays pledged.

A rippled amendment that ties an XLS-66 **Loan** to the token **Escrow** holding its collateral. While the loan is unpaid, `EscrowFinish` and `EscrowCancel` on that escrow return `tecPLEDGED` for every account. After payoff the escrow releases normally; on default the same `LoanManage` sweeps the collateral to the broker owner.

How to read the labels

PUBLIC DEVNET · anyone can check	On public XRPL Devnet: open the hash in any explorer.
SOURCE · GIT	In the git branch and patch files: hashes reproduce with git.
TEST LOGS	Test runs on the branch build.
OWN NODE · standalone · hashes local	Our standalone node at build 5df6bfbc9. Hashes exist only there; full JSON is in evidence/.
OWN NETWORK · 2 validators · hashes local	Our own two-validator network (network_id 424). Hashes exist only there; both nodes' ledger hashes are recorded.
EMULATOR · Speculos	Ledger's Speculos emulator running our app build. The physical-device run is pending.

Branch
collateral-lock @ 5df6bfbc9
6 commits on rippled tag 3.4.1

Tests
7 cases · 687 checks · 0 failures
+ 40-suite regression, 934 gtest

Own node
tecPLEDGED ×3, then release, then sweep
run 3, ledgers 52–100

Two validators
23 / 23 transactions agree
same validated ledger hash on both nodes

What to check, and where

Each claim, the evidence for it, and who can verify it. Public Devnet hashes anyone can open; own-node and own-network hashes are backed by the full JSON in evidence/.

Claim	Evidence	Where	Kind
The code is small and reviewable	6 commits on tag 3.4.1; 24 files, +1,005 / −2	Patch files, commit and tree hashes (p. 3–7)	SOURCE · GIT
It doesn't break existing behaviour	CollateralLock suite 7/687/0; 40 regression suites 2,279 cases / 253,852 checks, 0 failures; gtest 934 passed	Test logs (p. 8)	TEST LOGS
The problem is real today	Three successful Devnet transactions that release or take collateral the loan didn't agree to (two from Sat Oct 24, one pre-event context run)	Explorer screenshots (p. 9)	PUBLIC DEVNET · anyone can check
The ledger refuses release while the loan is unpaid	Finish (borrower), Finish (broker owner), Cancel (borrower) → tecPLEDGED	Run 3 terminal + hash table (p. 10–11)	OWN NODE · standalone · hashes local
Payoff releases; default sweeps	Same Finish after 3 LoanPay → tesSUCCESS; tfLoanDefault → 5,000 g to the broker owner	Run 3 (p. 11)	OWN NODE · standalone · hashes local
The borrower sees what they pledge	The Ledger app shows "Collateral ID" = the escrow key before signing the LoanSet	Device screens + review.json (p. 12)	EMULATOR · Speculos
Same rules under consensus	Two validators, quorum 2: every tx in the same validated ledger with the same ledger hash on both nodes	Crosscheck table + clip frame (p. 13–14)	OWN NETWORK · 2 validators · hashes local
Still working this morning	Live 'show me' run on the two-validator network, 23/23 agree	p. 15	OWN NETWORK · 2 validators · hashes local

Source register

Everything needed to rebuild exactly what we ran. Apply the patches to tag 3.4.1 with `git am`; the tree hash must match.

Commits on top of rippled 3.4.1

#	Commit (full)	Subject
1	6640cf5669855676b4bcc32d2edad3dacd168624	feat: Add the CollateralLock amendment, Loan.CollateralID and tecPLEDGED
2	384cf60a0d8378034f495019dfd306ccd0b53910	feat: EscrowFinish and EscrowCancel return tecPLEDGED while the loan is unpaid
3	5799210672ec90dbdfb2291945b4f858128bdb33	feat: LoanSet pledges a token escrow with CollateralID
4	6383866c9d35d19cbdb922ee36efb34b743cfc82	feat: LoanManage default sweeps pledged collateral to the broker owner
5	4273af4167712b5d64c14128c3750e0f6fdead25	feat: Add CollateralLock invariants
6	5df6bfbc91a6bc546444e22e0028ce96310b8f92	test: Add CollateralLock jtx suite

Base tag 3.4.1	00e640751460c137636fdb6dae6b4280de355d81
Branch tree	fe497140150f7f4c2455a36594d8d2c7d9db0896
Diff vs 3.4.1	24 files changed, 1,005 insertions, 2 deletions: 17 hand-written protocol/transactor files +383 / −2, 3 generated wrapper headers +107, 4 test files +515
Author / dates	Harry Malinski · all six commits 2026-10-24T22:53:19Z (squashed from the day's working tree)

Patch files (git format-patch)

File	sha256
0001-feat-Add-the-CollateralLock-amendment-Loan.Collatera.patch	203137fecb71987d19e637b451e07688ecc96314d84286fb56570a04a3642506
0002-feat-EscrowFinish-and-EscrowCancel-return-tecPLEDGED.patch	4dbdd0fc97a7506b4c14ca951be581fa2cf92f152d4b5978d9f8a0e021458854
0003-feat-LoanSet-pledges-a-token-escrow-with-Collaterall.patch	f7ce4164fbd358d57f687576c712d655fc20213cba06a97cc66cfed02e66f046
0004-feat-LoanManage-default-sweeps-pledged-collateral-to.patch	e5dfb9f19004798a046dc948e8a309cd2c5ae2e938b45c5ac512a2794f46e2cf
0005-feat-Add-CollateralLock-invariants.patch	a6ee7778d30908e1ad66ca9a3bc74a3c0c56f96a477d43b4132e43c2479e7e62
0006-test-Add-CollateralLock-jtx-suite.patch	e3466f4b50aafc3c1bf0bac92f66e35c97a8d179f9b0c687f460666bc57bb64c

Binaries

Two-validator xrpld (both nodes)	83b2fd254bac0a8eb47f132e87388a40f23a8c3cc23475fec23106ca1009c5ea
Version string	3.4.1 · `9ce853322` = same tree `fe497140` as 5df6bfbc9 (built before the commit messages were edited)
Ledger app, ClearPledge collateral-id: published app hash	8360d774d098a5093407a04187076b5bf26991d9032cad0ee70301d89395daaf
…app.elf file sha256	dc879fdc900cccb034b308a7472383df5e8a92436ddca80ea12b67ebab9c4432

The code I · amendment, field, result code

Patch 0001. One feature flag (default no), one optional UINT256 field on Loan and LoanSet, the existing LoanID reused on Escrow, one new tec code.

Amendment flag — features.macro · patch 0001

```
@@ -15,6 +15,7 @@
 // Add new amendments to the top of this list.
 // Keep it sorted in reverse chronological order.

+XRPL_FEATURE(CollateralLock,              Supported::Yes, VoteBehavior::DefaultNo)
 XRPL_FIX    (BatchV1_2,                   Supported::Yes, VoteBehavior::DefaultYes)
 XRPL_FIX    (Cleanup3_4_0,                Supported::Yes, VoteBehavior::DefaultNo)
 XRPL_FEATURE(Sponsor,                     Supported::Yes, VoteBehavior::DefaultNo)
```

Field: sfCollateralID, UINT256 43 — sfields.macro · patch 0001

```
@@ -211,6 +211,8 @@ TYPED_SFIELD(sfLoanID,                    UINT256,   38)
 TYPED_SFIELD(sfReferenceHolding,          UINT256,   39)
 TYPED_SFIELD(sfBlindingFactor,            UINT256,   40)
 TYPED_SFIELD(sfObjectID,                  UINT256,   41)
+//                                                    42 unused
+TYPED_SFIELD(sfCollateralID,              UINT256,   43)

 // number (common)
 TYPED_SFIELD(sfNumber,                    NUMBER,     1)
```

tecPLEDGED — include/xrpl/protocol/TER.h · patch 0001

```
@@ -370,6 +370,7 @@ enum TECcodes : TERUnderlyingType {
     tecNO_DELEGATE_PERMISSION = 198,
     tecBAD_PROOF = 199,
     tecNO_SPONSOR_PERMISSION = 200,
+    tecPLEDGED = 201,
 };

 //------------------------------------------------------------------------------
```

Formats: Loan and Escrow — ledger_entries.macro · patch 0001

```
@@ -347,6 +347,7 @@ LEDGER_ENTRY(ltESCROW, 0x0075, Escrow, escrow, ({
     {sfDestinationNode,      SoeOptional},
     {sfTransferRate,         SoeOptional},
     {sfIssuerNode,           SoeOptional},
+    {sfLoanID,               SoeOptional},
 }))

 /** A ledger object describing a single unidirectional XRP payment channel.
```

```
@@ -565,6 +566,7 @@ LEDGER_ENTRY(ltLOAN, 0x0089, Loan, loan, ({
     {sfGracePeriod,          SoeDefault},
     {sfPreviousPaymentDueDate, SoeDefault},
     {sfNextPaymentDueDate,   SoeDefault},
+    {sfCollateralID,         SoeOptional},
     // The loan object tracks these values:
     //
     // - PaymentRemaining: The number of payments left in the loan. When it
```

Format: LoanSet takes CollateralID — transactions.macro · patch 0001

```
@@ -983,6 +983,7 @@ TRANSACTION(ttLOAN_SET, 80, LoanSet,
     {sfPaymentTotal, SoeOptional},
     {sfPaymentInterval, SoeOptional},
     {sfGracePeriod, SoeOptional},
+    {sfCollateralID, SoeOptional},
 }))

 /** This transaction deletes an existing Loan */
```

The code II · the gate on EscrowFinish and EscrowCancel

Patch 0002. A pledged escrow is refused while its loan has payments remaining or is defaulted; a missing loan means not pledged.

isPledged / canSweepCollateral — EscrowHelpers.h · patch 0002

```
 namespace xrpl {

+/** Returns true while the escrow backs a loan that is still owed.
+
+    An escrow pledged by LoanSet (CollateralLock) carries the loan's ID in
+    sfLoanID. It stays pledged until the loan is paid off, or until the loan is
+    deleted after payoff. A defaulted loan's escrow is swept by LoanManage, so
+    a defaulted loan that still has its escrow keeps it pledged.
+*/
+inline bool
+isPledged(ReadView const& view, SLE const& escrow)
+{
+    auto const loanID = escrow[~sfLoanID];
+    if (!loanID)
+        return false;
+
+    auto const loan = view.read(keylet::loan(*loanID));
+    if (!loan)
+        return false;
+
+    return loan->at(sfPaymentRemaining) != 0 || loan->isFlag(lsfLoanDefault);
+}
+
+/** Checks that a pledged escrow's collateral can move to `receiver`.
+
+    LoanManage sweeps a defaulted loan's collateral from the borrower to the
+    loan broker's owner. LoanSet runs the same check when the escrow is
+    pledged, so a loan is never written against collateral that could not be
+    swept.
+*/
+inline TER
+canSweepCollateral(ReadView const& view, SLE const& escrow, AccountID const& receiver)
+{
+    STAmount const amount = escrow[sfAmount];
+    Asset const asset = amount.asset();
+    AccountID const owner = escrow[sfAccount];
+
+    if (amount.getIssuer() == receiver)
+        return tesSUCCESS;
+
+    if (!holdingExists(view, receiver, asset))
+        return tecNO_AUTH;
+    if (auto const ret = requireAuth(view, asset, receiver, AuthType::WeakAuth))
+        return ret;
… (continues in the patch file)
```

EscrowFinish preclaim — EscrowFinish.cpp · patch 0002

```
@@ -202,6 +202,17 @@ EscrowFinish::preclaim(PreclaimContext const& ctx)
             return err;
     }

+    if (ctx.view.rules().enabled(featureCollateralLock))
+    {
+        auto const k =
+            keylet::escrow(ctx.tx[sfOwner], SeqProxy::rawSequence(ctx.tx[sfOfferSequence]));
+        if (auto const slep = ctx.view.read(k); slep && isPledged(ctx.view, *slep))
+        {
+            JLOG(ctx.j.trace()) << "Escrow is pledged to an unpaid loan.";
+            return tecPLEDGED;
+        }
+    }
+
     if (ctx.view.rules().enabled(featureTokenEscrow))
     {
         auto const seqProxy = SeqProxy::rawSequence(ctx.tx[sfOfferSequence]);
```

EscrowCancel preclaim — EscrowCancel.cpp · patch 0002

```
@@ -91,6 +91,17 @@ escrowCancelPreclaimHelper<MPTIssue>(
 TER
 EscrowCancel::preclaim(PreclaimContext const& ctx)
 {
+    if (ctx.view.rules().enabled(featureCollateralLock))
+    {
+        auto const k =
+            keylet::escrow(ctx.tx[sfOwner], SeqProxy::rawSequence(ctx.tx[sfOfferSequence]));
+        if (auto const slep = ctx.view.read(k); slep && isPledged(ctx.view, *slep))
+        {
+            JLOG(ctx.j.trace()) << "Escrow is pledged to an unpaid loan.";
+            return tecPLEDGED;
+        }
+    }
+
     if (ctx.view.rules().enabled(featureTokenEscrow))
     {
         auto const seqProxy = SeqProxy::rawSequence(ctx.tx[sfOfferSequence]);
```

The code III · LoanSet pledges the escrow

Patch 0003. Preflight and preclaim checks (escrow exists, token escrow, self-escrow, not already pledged, CancelAfter ≥ maturity, sweep possible), then the two back-pointers.

checkCollateral: the six checks

LoanSet.cpp · patch 0003

```
+
+TER
+LoanSet::checkCollateral(
+    ReadView const& view,
+    uint256 const& collateralID,
+    AccountID const& borrower,
+    AccountID const& brokerOwner,
+    std::uint64_t maturity,
+    beast::Journal j)
+{
+    auto const escrow = view.read(Keylet{ltESCROW, collateralID});
+    if (!escrow)
+    {
+        JLOG(j.warn()) << "Collateral escrow does not exist.";
+        return tecNO_ENTRY;
+    }
+
+    STAmount const amount = (*escrow)[sfAmount];
+    if (isXRP(amount))
+    {
+        JLOG(j.warn()) << "Collateral must be a token escrow.";
+        return tecWRONG_ASSET;
+    }
+
+    // Only the borrower can pledge, and only to themselves: after payoff the
+    // collateral can go nowhere but back to the borrower.
+    if ((*escrow)[sfAccount] != borrower || (*escrow)[sfDestination] != borrower)
+    {
+        JLOG(j.warn()) << "Collateral escrow must be owned by and payable to the borrower.";
+        return tecNO_PERMISSION;
+    }
+
+    if (escrow->isFieldPresent(sfLoanID))
+    {
+        JLOG(j.warn()) << "Collateral escrow is already pledged.";
+        return tecDUPLICATE;
+    }
+
+    // The borrower must not be able to cancel before the loan can default.
+    if (auto const cancelAfter = (*escrow)[~sfCancelAfter]; cancelAfter && *cancelAfter < maturity)
+    {
+        JLOG(j.warn()) << "Collateral escrow can be cancelled before the loan matures.";
+        return tecTOO_SOON;
+    }
+
+    // The default sweep pays the broker owner, so it must already be able to
+    // receive the collateral.
+    if (auto const ret = canSweepCollateral(view, *escrow, brokerOwner))
+    {
+        JLOG(j.warn()) << "Collateral could not be swept to the broker owner.";
+        return ret;
+    }
… (continues in the patch file)
```

preflight and preclaim

LoanSet.cpp · patch 0003

```
 #include <xrpl/ledger/helpers/SponsorHelpers.h>
 #include <xrpl/ledger/helpers/TokenHelpers.h>
@@ -149,6 +150,14 @@ LoanSet::preflight(PreflightContext const& ctx)
     if (auto const brokerID = ctx.tx[~sfLoanBrokerID]; brokerID && *brokerID == beast::kZero)
         return temINVALID;

+    if (auto const collateralID = tx[~sfCollateralID])
+    {
+        if (!ctx.rules.enabled(featureCollateralLock))
+            return temDISABLED;
+        if (*collateralID == beast::kZero)
+            return temINVALID;
+    }
+
     return tesSUCCESS;
 }

@@ -424,6 +433,71 @@ LoanSet::preclaim(PreclaimContext const& ctx)
         return ret;
     }

+    if (auto const collateralID = tx[~sfCollateralID])
+    {
+        auto const grace = tx[~sfGracePeriod].value_or(kDefaultGracePeriod);
… (continues in the patch file)
```

doApply: the two back-pointers

LoanSet.cpp · patch 0003

```
 }

@@ -689,6 +763,16 @@ LoanSet::doApply()
     loan->at(sfPreviousPaymentDueDate) = 0;
     loan->at(sfNextPaymentDueDate) = startDate + paymentInterval;
     loan->at(sfPaymentRemaining) = paymentTotal;
+    if (auto const collateralID = tx[~sfCollateralID])
+    {
+        // Pledge the escrow: the loan points at it, and it points back.
+        auto const escrow = view.peek(Keylet{ltESCROW, *collateralID});
+        if (!escrow)
+            return tefBAD_LEDGER;  // LCOV_EXCL_LINE
+        loan->setFieldH256(sfCollateralID, *collateralID);
+        escrow->setFieldH256(sfLoanID, loan->key());
+        view.update(escrow);
+    }
… (continues in the patch file)
```

The code IV · default sweep and invariant

Patches 0004 and 0005. On tfLoanDefault the collateral moves to the broker owner with the same helper EscrowFinish uses; a new invariant guards LoanID and pledged-escrow deletion.

LoanManage.cpp: sweepCollateral — LoanManage.cpp · patch 0004

```
+
+TER
+LoanManage::sweepCollateral(SLE::ref loanSle, SLE::ref brokerSle)
+{
+    auto const collateralID = (*loanSle)[~sfCollateralID];
+    if (!collateralID)
+        return tesSUCCESS;
+
+    auto& view = ctx_.view();
+    Keylet const k{ltESCROW, *collateralID};
+    auto const slep = view.peek(k);
+    if (!slep)
+        return tefBAD_LEDGER;  // LCOV_EXCL_LINE
+
+    AccountID const borrower = (*slep)[sfAccount];
+    AccountID const brokerOwner = brokerSle->at(sfOwner);
+    auto const sleOwner = view.peek(keylet::account(brokerOwner));
+    if (!sleOwner)
+        return tefBAD_LEDGER;  // LCOV_EXCL_LINE
+
+    // Remove the escrow from every directory that lists it.
+    if (!view.dirRemove(keylet::ownerDir(borrower), (*slep)[sfOwnerNode], k.key, true))
+        return tefBAD_LEDGER;  // LCOV_EXCL_LINE
+    if (auto const optPage = (*slep)[~sfDestinationNode];
+        optPage && !view.dirRemove(keylet::ownerDir((*slep)[sfDestination]), *optPage, k.key, true))
+        return tefBAD_LEDGER;  // LCOV_EXCL_LINE
+
+    STAmount const amount = (*slep)[sfAmount];
+    auto const issuer = amount.getIssuer();
+    if (auto const optPage = (*slep)[~sfIssuerNode];
+        optPage && !view.dirRemove(keylet::ownerDir(issuer), *optPage, k.key, true))
+        return tefBAD_LEDGER;  // LCOV_EXCL_LINE
+
+    // Deliver the collateral to the broker owner, exactly as an EscrowFinish
+    // with the broker owner as the destination would.
+    Rate const lockedRate = slep->isFieldPresent(sfTransferRate)
+        ? Rate(slep->getFieldU32(sfTransferRate))
+        : kParityRate;
+    if (auto const ret = std::visit(
+            [&]<typename T>(T const&) {
+                return escrowUnlockApplyHelper<T>(
+                    ctx_.getApplyViewContext(),
+                    lockedRate,
+                    sleOwner,
+                    sleOwner->at(sfBalance).value().xrp(),
+                    amount,
+                    issuer,
+                    borrower,
+                    brokerOwner,
+                    false,
+                    j_);
+            },
… (continues in the patch file)
```

ValidCollateralLock — LoanInvariant.cpp · patch 0005

```
@@ -295,4 +295,74 @@ ValidLoan::finalize(
     return true;
 }

+void
+ValidCollateralLock::visitEntry(bool isDelete, SLE::const_ref before, SLE::const_ref after)
+{
+    if (isDelete)
+    {
+        if (before && before->getType() == ltESCROW && before->isFieldPresent(sfLoanID))
+            deletedPledges_.push_back(before);
+        return;
+    }
+
+    if (!after)
+        return;  // LCOV_EXCL_LINE
+
+    if (after->getType() == ltESCROW && before && (*before)[~sfLoanID] != (*after)[~sfLoanID])
+        changedPledges_.emplace_back(before, after);
+
+    if (after->getType() == ltLOAN && !before && after->isFieldPresent(sfCollateralID))
+        newLoans_.push_back(after);
+}
+
+bool
+ValidCollateralLock::finalize(
+    STTx const& tx,
+    TER const result,
+    XRPAmount const,
+    ReadView const& view,
+    beast::Journal const& j)
+{
+    if (!isTesSuccess(result))
+        return true;
+
+    auto const txType = tx.getTxnType();
+
+    for (auto const& [before, after] : changedPledges_)
+    {
+        if (txType != ttLOAN_SET || before->isFieldPresent(sfLoanID) ||
+            !after->isFieldPresent(sfLoanID))
+        {
+            JLOG(j.fatal()) << "Invariant failed: Escrow LoanID changed outside LoanSet";
+            return false;
+        }
+    }
+
+    for (auto const& escrow : deletedPledges_)
+    {
+        auto const loan = view.read(keylet::loan((*escrow)[sfLoanID]));
+        bool const paidOff =
+            !loan || (loan->at(sfPaymentRemaining) == 0 && !loan->isFlag(lsfLoanDefault));
… (continues in the patch file)
```

Tests

Run on the branch build. The CollateralLock suite, then the existing suites most likely to be affected, then gtest including the regenerated protocol wrappers.

$ ./xrpld --unittest=CollateralLock

```
xrpl.tx.CollateralLock Disabled
xrpl.tx.CollateralLock Loan without CollateralID is unchanged
xrpl.tx.CollateralLock LoanSet preclaim
xrpl.tx.CollateralLock Finish and Cancel return tecPLEDGED until payoff
xrpl.tx.CollateralLock EscrowCancel succeeds after payoff and LoanDelete
xrpl.tx.CollateralLock Default sweeps the collateral to the broker owner
xrpl.tx.CollateralLock Default fails, and the pledge stays, if the sweep is impossible
xrpl.tx.CollateralLock had 0 failures.
1.0s, 1 suite, 7 cases, 687 tests total, 0 failures
```

$ ./xrpld --unittest=Escrow,EscrowToken,Loan,LendingHelpers,Vault,Invariants,Feature,TER,MPToken,Batch (last lines)

```
   36.3s xrpl.tx.LoanLifecycle
   24.2s xrpl.tx.LoanSet
   21.7s xrpl.app.VaultBugs
   19.3s xrpl.tx.LoanSecurity
   19.2s xrpl.tx.LoanInvariants
   18.3s xrpl.app.Batch
   18.2s xrpl.app.VaultLifecycle
   15.4s xrpl.app.InvariantsVault
78.0s, 40 suites, 2279 cases, 253852 tests total, 0 failures
```

$ ./xrpl_tests (gtest, summary)

```
[==========] 934 tests from 201 test suites ran. (16306 ms total)
[  PASSED  ] 934 tests.
  YOU HAVE 3 DISABLED TESTS
```

The seven test cases (src/test/app/lending/CollateralLock_test.cpp)

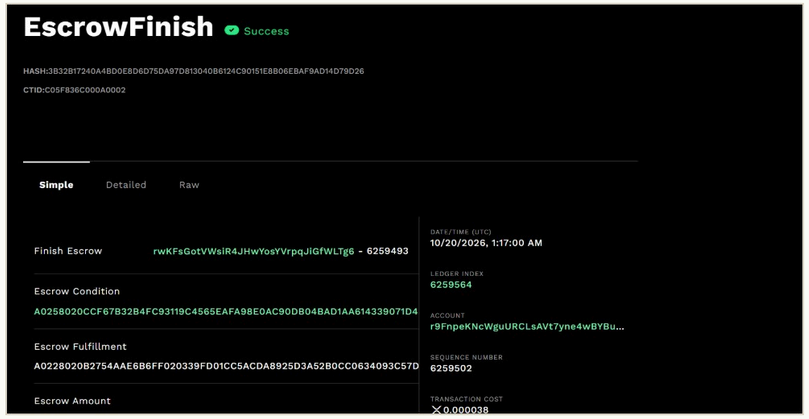
1. Disabled
2. Loan without CollateralID is unchanged
3. LoanSet preclaim
4. Finish and Cancel return tecPLEDGED until payoff
5. EscrowCancel succeeds after payoff and LoanDelete
6. Default sweeps the collateral to the broker owner
7. Default fails, and the pledge stays, if the sweep is impossible

> Coverage gap, stated plainly: every case pledges an MPT. The IOU (trust-line) path runs through the same shared helpers but has no dedicated case yet.

The problem on public Devnet (stock rippled 3.4.1, no amendment)

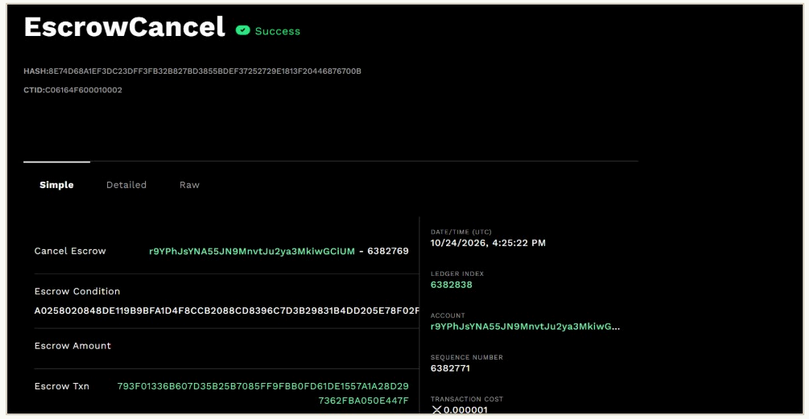
PUBLIC DEVNET · anyone can check

Three successful transactions where an escrow backing a loan is released or taken without the loan's consent. Anyone can open these on a Devnet explorer (until Devnet resets).



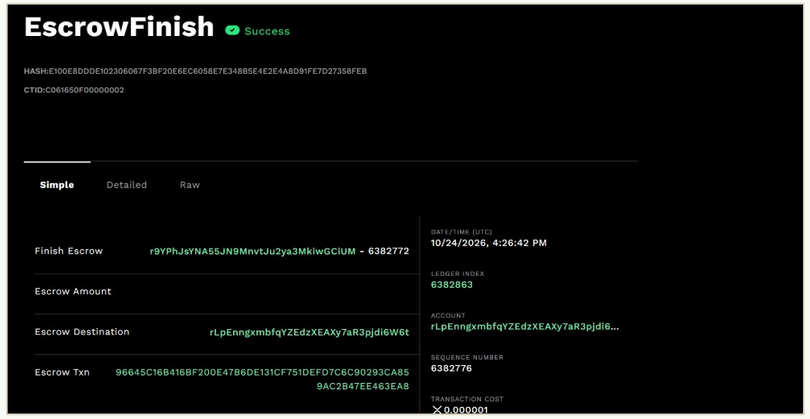
EscrowFinish releases collateral while its loan is unpaid (Oct 19 EDT, pre-event context run)

3B32B17240A4BD0E8D6D75DA97D813040B6124C90151E8B06EBAF9AD14D79D26



Borrower cancels the escrow after CancelAfter, loan unpaid

8E74D68A1EF3DC23DFF3FB32B827BD3855BDEF37252729E1813F20446876700B



Broker finishes a repaid loan's escrow after FinishAfter

E100E8DDDE102306067F3BF20E6EC6058E7E348B5E4E2E4A8D91FE7D27358FEB

> Also on Devnet: LoanSet 8BA81F689A09613D1BA190AF0530F25EC7052B100FAC598FCDE70768EBC5ADEF, signed by the patched Ledger app in Speculos. It carries no CollateralID: Devnet runs stock 3.4.1, and the Collateral ID screen exists only in the build for our node.

Own node · run 3 · the terminal

The actual console output of proof_run_ledger.py against our standalone node at build 5df6bfbc9 (evidence/proof_run_ledger_terminal.raw). The borrower signs every transaction on the Ledger app in Speculos.

OWN NODE · standalone · hashes local

EMULATOR · Speculos


```
$ python3 proof_run_ledger.py http://127.0.0.1:5035
CollateralLock proof run  xrpld 3.4.1 · branch collateral-lock @ 5df6bfbc9 · standalone

setup: accounts, LGAU gram token (MPT), vault + loan broker
borrower = the Ledger app's account ra7Zr8ddy9tB88RaXL8B87YkqhEJG2vkAJ (Speculos · ClearPledge collateral-id build)
  fund rM3top7YvB…                                    tesSUCCESS
  fund ra7Zr8ddy9…                                    tesSUCCESS
  fund rJrfTYaFMN…                                    tesSUCCESS
  MPTokenIssuanceCreate LGAU (scale 3, CanEscrow + CanTransfer) tesSUCCESS
  ✍ signed on the Ledger app (Speculos · collateral-id build) · 8 screens · 12_MPTokenAuthorize
  MPTokenAuthorize ra7Zr8ddy9…                        tesSUCCESS
  MPTokenAuthorize rJrfTYaFMN…                        tesSUCCESS
  Payment 20,000 g LGAU to the borrower               tesSUCCESS
  VaultCreate (XRP)                                   tesSUCCESS
  VaultDeposit 50,000 XRP                             tesSUCCESS
  LoanBrokerSet (cover 10 %)                          tesSUCCESS
  LoanBrokerCoverDeposit 1,000 XRP                    tesSUCCESS

Loan A: 10,000 g LGAU pledged to a 1,000 XRP loan
  ✍ signed on the Ledger app (Speculos · collateral-id build) · 12 screens · 13_EscrowCreate
  EscrowCreate 10,000 g (self-escrow)                 tesSUCCESS
  ✍ signed on the Ledger app (Speculos · collateral-id build) · 16 screens · 14_LoanSet

LoanSet with CollateralID = the escrow
{
 "engine_result": "tesSUCCESS",
 "TransactionResult": "tesSUCCESS",
 "ledger_index": 64,
 "hash": "FEC09B554FDA7CA26EE992F2361B1D29F3619D71EDCB2BB8F4AC13A6C2B27D63"
}
  Loan.CollateralID = 9B6002B543594CE456177DD215DE1E3EBA195C776DE87845D7417A282096BFA9
  Escrow.LoanID     = 04F12C107D8AEFCE4C078B123D10FED99CCC3752F63863ED8A1AA4CE4BCADA23
  ✍ signed on the Ledger app (Speculos · collateral-id build) · 9 screens · 15_EscrowFinish

EscrowFinish, loan open (borrower)
{
 "engine_result": "tecPLEDGED",
 "TransactionResult": "tecPLEDGED",
 "ledger_index": 66,
 "hash": "A7A71A7D4AA6A94DA076C3DADC26BD1E250F47C51C55D04863B0BFD61A3CF25E"
}

EscrowFinish, loan open (broker owner)
{
 "engine_result": "tecPLEDGED",
 "TransactionResult": "tecPLEDGED",
 "ledger_index": 67,
 "hash": "D2249D9BB7D4B1C3972C77D4608EC20D3D446BF7739DBE9F6BE6EB0090247D35"
}
  ✍ signed on the Ledger app (Speculos · collateral-id build) · 9 screens · 16_EscrowCancel

EscrowCancel, loan open (borrower)
{
 "engine_result": "tecPLEDGED",
 "TransactionResult": "tecPLEDGED",
 "ledger_index": 68,
 "hash": "5CDFEFA5612FEB387BFB605A717F7ADF9436A01C12D86ECCB477DD8B7536279D"
}
  ✍ signed on the Ledger app (Speculos · collateral-id build) · 10 screens · 17_LoanPay
  LoanPay 1/3 (333.333334 XRP)                        tesSUCCESS
  ✍ signed on the Ledger app (Speculos · collateral-id build) · 10 screens · 18_LoanPay
  LoanPay 2/3 (333.333334 XRP)                        tesSUCCESS
  ✍ signed on the Ledger app (Speculos · collateral-id build) · 10 screens · 19_LoanPay
  LoanPay 3/3 (333.333334 XRP)                        tesSUCCESS
  loan PaymentRemaining = 0 (closed)
  ✍ signed on the Ledger app (Speculos · collateral-id build) · 9 screens · 20_EscrowFinish

EscrowFinish, loan paid off (borrower)
{
 "engine_result": "tesSUCCESS",
 "TransactionResult": "tesSUCCESS",
 "ledger_index": 72,
 "hash": "7B7FF9E84D01C3991CD00935A30BE3A76E458C4851C3E24049CD70BE5929B165"
}
  escrow: gone · borrower LGAU 20,000 g

Loan B: 5,000 g LGAU pledged, then the loan defaults
  ✍ signed on the Ledger app (Speculos · collateral-id build) · 12 screens · 21_EscrowCreate
  EscrowCreate 5,000 g (self-escrow)                  tesSUCCESS
  ✍ signed on the Ledger app (Speculos · collateral-id build) · 16 screens · 22_LoanSet

LoanSet with CollateralID = the escrow
{
 "engine_result": "tesSUCCESS",
 "TransactionResult": "tesSUCCESS",
 "ledger_index": 74,
 "hash": "5FCC29B7F69184CCB04D49A33D68AC41B3BCD68482A2C462581827B5254853AB"
}
  waiting for NextPaymentDueDate + GracePeriod (120 s)…

LoanManage tfLoanDefault (broker owner)
{
 "engine_result": "tesSUCCESS",
 "TransactionResult": "tesSUCCESS",
 "ledger_index": 100,
 "hash": "72D5D727D55C5CA92D062BD8F1600FC7ADF85202C324F9467B3A97FF1C68B268"
}
  escrow: gone · broker owner received 5,000 g LGAU

saved /root/rec2/proof_rec.json
```

Own node · run 3 · every transaction

All 23 validated transactions, read from evidence/proof_run_result.json (build 5df6bfbc9). Highlighted rows are the ones the amendment decides.

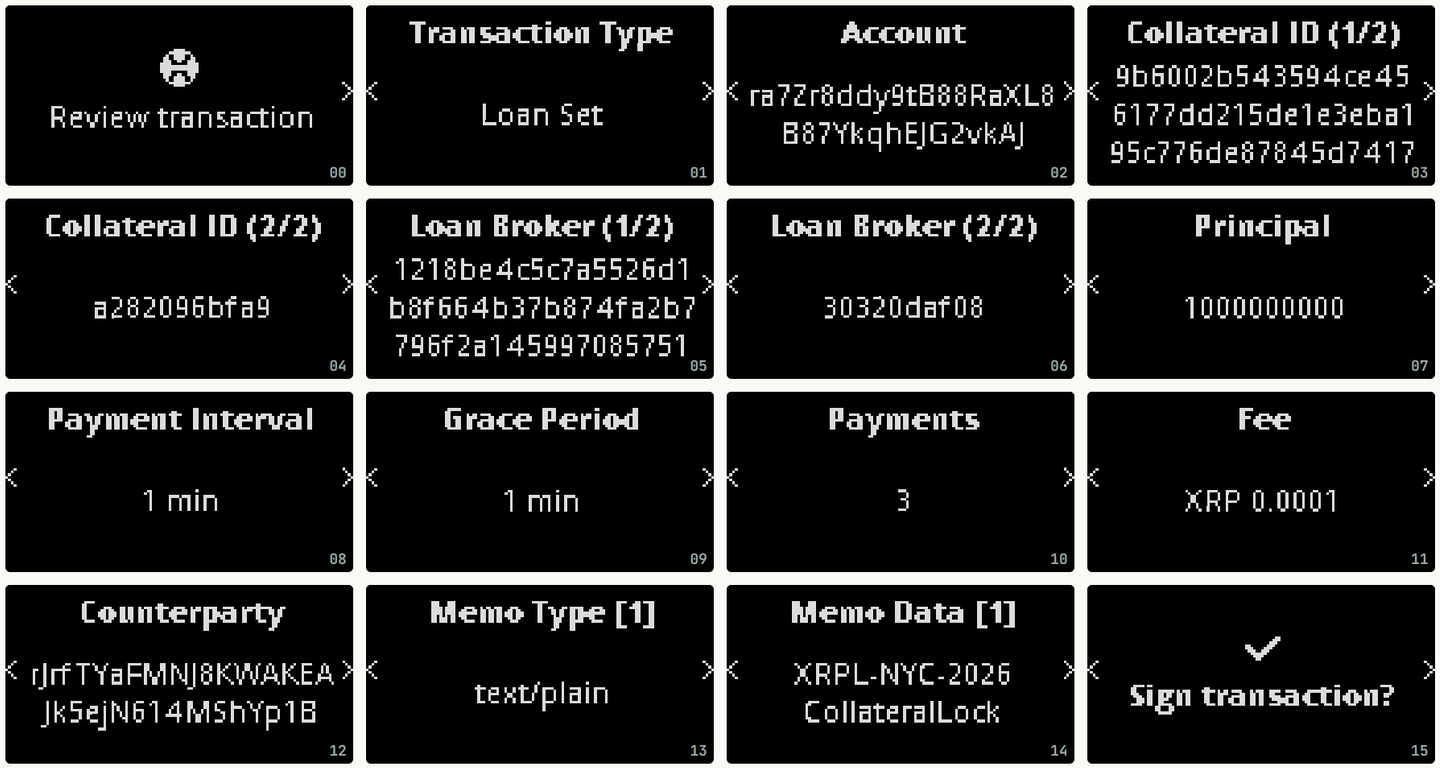
Ledger	Type	Result	Hash (full)
52	Payment	tesSUCCESS	D4E2A2ABEE40ABA660AAD8ACB033B16FA7BE5C1DA2A61228AB9C605148E73E71
53	Payment	tesSUCCESS	5B21DD1FC0AC49084F0ABB5E07CF3C00EB30155985120AC8D3B2244E0A9A380A
54	Payment	tesSUCCESS	9453C62681FEC47B9657BD335B7C6833F2D0DC6B8DEEFEA2D62FA2A508603BA2
55	MPTokenIssuanceCreate	tesSUCCESS	49B6EF4ACC689CFC7C586C7872AE5956210348DD45D46042FB7F75FD23CB26A2
56	MPTokenAuthorize	tesSUCCESS	BAAD91BF0327280745F0C71E0846C6B4FF404AB030BBEA38F7D74CA0DF1443A6
57	MPTokenAuthorize	tesSUCCESS	A2B562AA486DF66A3E71BC53FE067869C7AFBEAFADC0E59BBE22DB0ABC33C642
58	Payment	tesSUCCESS	2952F8982DADE6ECAA692AC6C4AACD2C3A1EF5D27425724E507927116CB64EF0
59	VaultCreate	tesSUCCESS	DE76C46EEFBFA0DC9B3E3A00699140EC1D2EB73C42C8328704191E2ADBB5382F
60	VaultDeposit	tesSUCCESS	206430FCABF8E3A3ED266B8A4F5AE0CA3AEE3E05B5733C058F18713BFECB1424
61	LoanBrokerSet	tesSUCCESS	C70CEBE47AB3EB34CD380B70D21EE0DB6C508596766C85D4A37ECA0E5733AA8E
62	LoanBrokerCoverDeposit	tesSUCCESS	ED03368FBD115BCA8E7AE72A42591FA4B98D2E9B00A4F3016355F9F6322CC848
63	EscrowCreate	tesSUCCESS	3A105FC171F05A593611BE1CA24D4D0F755B241133F317A1B61B4661305EF83C
64	LoanSet	tesSUCCESS	FEC09B554FDA7CA26EE992F2361B1D29F3619D71EDCB2BB8F4AC13A6C2B27D63
66	EscrowFinish	tecPLEDGED	A7A71A7D4AA6A94DA076C3DADC26BD1E250F47C51C55D04863B0BFD61A3CF25E
67	EscrowFinish	tecPLEDGED	D2249D9BB7D4B1C3972C77D4608EC20D3D446BF7739DBE9F6BE6EB0090247D35
68	EscrowCancel	tecPLEDGED	5CDFEFA5612FEB387BFB605A717F7ADF9436A01C12D86ECCB477DD8B7536279D
69	LoanPay	tesSUCCESS	95122DC0CB3282BBEF8611AEA3D63476C0051B17EC60AD5E1A7A3CD7930D90E3
70	LoanPay	tesSUCCESS	7993EEDC81DCF95FAC0A3D243B0438EA7E0FB3C3C8F7F624A460FEF5F37B3614
71	LoanPay	tesSUCCESS	48514F6B5F266579B119585DD8B3D0C3222637084A2EF621050BCB2292BF79F9
72	EscrowFinish	tesSUCCESS	7B7FF9E84D01C3991CD00935A30BE3A76E458C4851C3E24049CD70BE5929B165
73	EscrowCreate	tesSUCCESS	3C0A58AE0BB5AF7CBB930711BCAE486DA0EED6914CCD17B0716E1A245BB56901
74	LoanSet	tesSUCCESS	5FCC29B7F69184CCB04D49A33D68AC41B3BCD68482A2C462581827B5254853AB
100	LoanManage	tesSUCCESS	72D5D727D55C5CA92D062BD8F1600FC7ADF85202C324F9467B3A97FF1C68B268

Loan A / its escrow (CollateralID)	04F12C107D8AEFCE4C078B123D10FED99CCC3752F63863ED8A1AA4CE4BCADA23 / 9B6002B543594CE456177DD215DE1E3EBA195C776DE87845D7417A282096BFA9
Loan B / its escrow (swept on default)	99803617E6FFACFBA808E04E08B9A8CC2FB97E82E906F13659D3D1ED9946C246 / E366C5B5B583CB31D8048E1F70735CDACE50FE616937F7C4F2A4474E7FA8168A
Accounts	borrower ra7Zr8ddy9tB88RaXL8B87YkqhEJG2vkAJ (Ledger app in Speculos) · broker owner rJrfTYaFMNJ8KWAKEAJk5ejN614MShYp1B · issuer rM3top7YvBZfGsjDfWCBQ6LKcMD45twvZA

The borrower sees the pledge before signing

Loan A's LoanSet as reviewed on the Ledger XRP app (ClearPledge collateral-id build) in Speculos: every screen, in order (proof/device_screens/14_LoanSet).

EMULATOR · Speculos

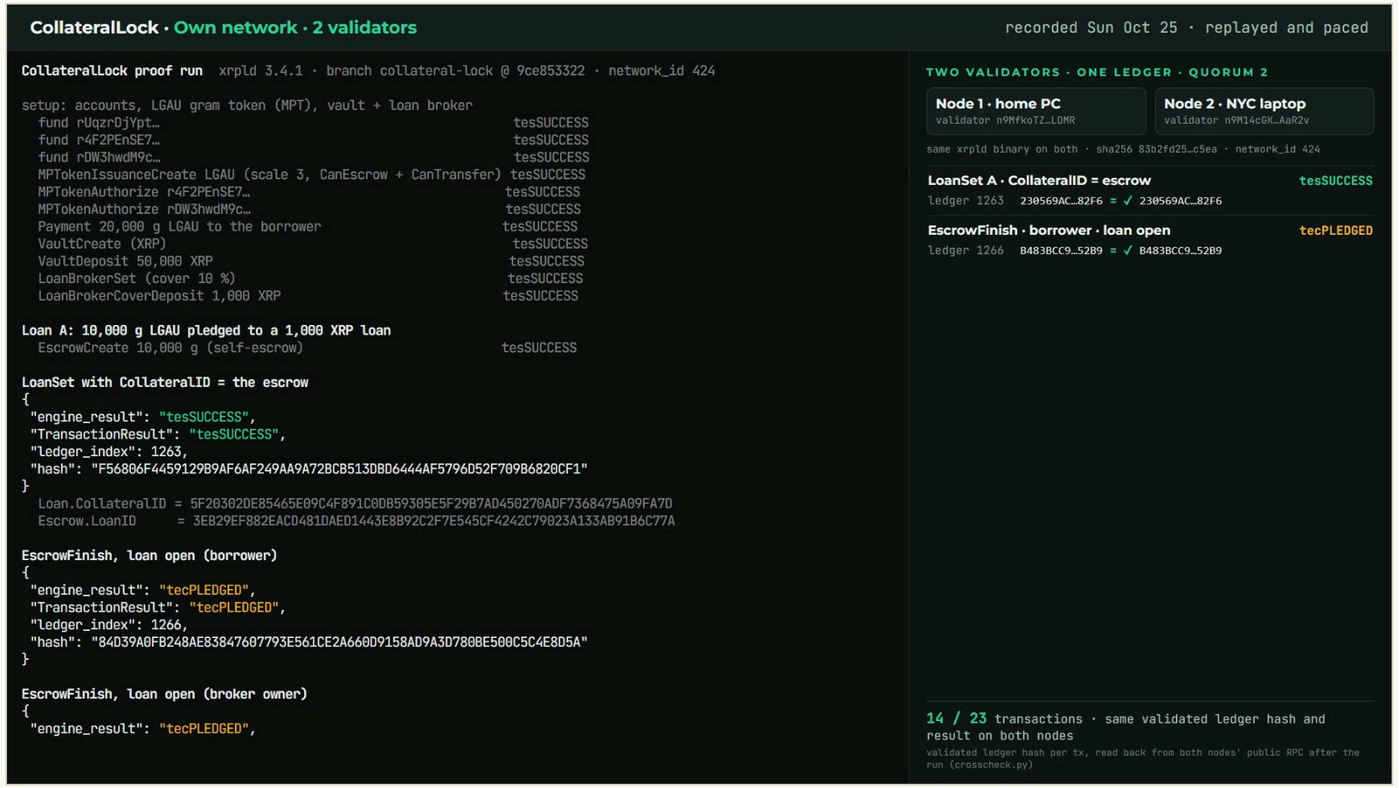


Screen 03–04 "Collateral ID"	`9B6002B543594CE456177DD215DE1E3EBA195C776DE87845D7417A282096BFA9`
= loan A's escrow key in the run JSON	`9B6002B543594CE456177DD215DE1E3EBA195C776DE87845D7417A282096BFA9`
Match	**yes**
Signing account	`ra7Zr8ddy9tB88RaXL8B87YkqhEJG2vkAJ`
Resulting LoanSet	`FEC09B554FDA7CA26EE992F2361B1D29F3619D71EDCB2BB8F4AC13A6C2B27D63 (ledger 64, tesSUCCESS)`
App	`app hash 8360d774d098a509… · app-xrp branch lending-clear-sign @ 79bcda9`

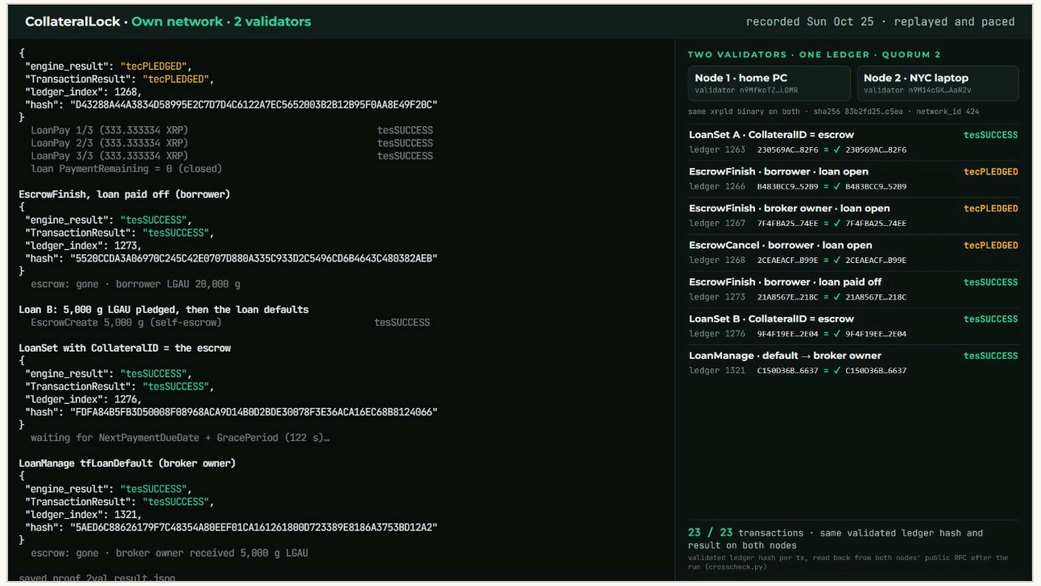
> The stock Ledger XRP app refuses a LoanSet with 0x680B before showing any screen. This build adds the XLS-65/66 types and field labels, including Collateral ID, so the borrower reads what they are pledging. Emulator only: the physical Nano S Plus run is pending, and "Nano S Plus" is said only after a hash from the device address is verified on-ledger.

Own network · two validators · the recording

Recorded Oct 25 03:42–03:45 UTC (Sat Oct 24, 23:42–23:45 EDT). Two validators (home PC, NYC laptop), network_id 424, quorum 2, both running the same xrpld binary. This is the run in the pitch deck's two-validator clip (CollateralLock_2val.mp4).



Frame at 9.6 s of CollateralLock_2val.mp4: left, the run; right, each key transaction's validated ledger hash read back from both nodes.



Frame at 23.5 s: the default sweep, and the running count of agreeing transactions.

xrpld sha256 (both nodes)	`83b2fd254bac0a8eb47f132e87388a40f23a8c3cc23475fec23106ca1009c5ea`
Ledgers	1250–1321
Agreement	**23 / 23** transactions, all_agree = true

Own network · two validators · crosscheck

For each transaction: its result, the validated ledger, and the ledger hash each node reports (evidence/proof_2val_crosscheck.json). Equal hashes mean both validators accepted the same ledger.

Ledger	Type	Result	Tx hash	Ledger hash · node 1 (PC)	node 2 (laptop)	=
1250	Payment	tesSUCCESS	7F72833DD875…451E	2B324E879A0D…6EDE	2B324E879A0D…6EDE	✓
1252	Payment	tesSUCCESS	0E1A8AEF6E49…0CA5	6AB0ED6D1988…E583	6AB0ED6D1988…E583	✓
1253	Payment	tesSUCCESS	2AC8A24C77DC…A181	699E3DB3A54F…F2D5	699E3DB3A54F…F2D5	✓
1254	MPTokenIssuanceCreate	tesSUCCESS	05BD4279501F…5AD2	0B141F498287…B715	0B141F498287…B715	✓
1255	MPTokenAuthorize	tesSUCCESS	331A679A47E5…D60B	D8F95B9A3661…9AD6	D8F95B9A3661…9AD6	✓
1256	MPTokenAuthorize	tesSUCCESS	38C76AD31DDF…73A4	D30B9BF6FC74…B135	D30B9BF6FC74…B135	✓
1257	Payment	tesSUCCESS	DC1DE1556DB8…57D5	C8B5935ACB13…306F	C8B5935ACB13…306F	✓
1258	VaultCreate	tesSUCCESS	2D5D8CE5F06B…7753	57D2139082C1…62AF	57D2139082C1…62AF	✓
1259	VaultDeposit	tesSUCCESS	411DBF4ABC6C…F24B	1B29150DDBFF…A375	1B29150DDBFF…A375	✓
1260	LoanBrokerSet	tesSUCCESS	7E61BD89A641…2C2D	F15CE7B11AD2…6FDF	F15CE7B11AD2…6FDF	✓
1261	LoanBrokerCoverDeposit	tesSUCCESS	5AAA1AAAF8AA…B5CB	BD97EC4A3C0F…CAAF	BD97EC4A3C0F…CAAF	✓
1262	EscrowCreate	tesSUCCESS	8CC244A370A6…C893	1CB9506609F2…1548	1CB9506609F2…1548	✓
1263	LoanSet	tesSUCCESS	F56806F44591…0CF1	230569ACE82E…82F6	230569ACE82E…82F6	✓
1266	EscrowFinish	tecPLEDGED	84D39A0FB248…8D5A	B483BCC9DBD6…52B9	B483BCC9DBD6…52B9	✓
1267	EscrowFinish	tecPLEDGED	A47A7BBAC96F…AB17	7F4FBA25AFC1…74EE	7F4FBA25AFC1…74EE	✓
1268	EscrowCancel	tecPLEDGED	D43288A44A38…F20C	2CEAEACFC8A3…B99E	2CEAEACFC8A3…B99E	✓
1269	LoanPay	tesSUCCESS	3DD37D9D9D58…2529	D3E9C9B20349…D445	D3E9C9B20349…D445	✓
1270	LoanPay	tesSUCCESS	96DF691FB57B…D9AC	25162F296E56…5191	25162F296E56…5191	✓
1271	LoanPay	tesSUCCESS	9B868725AAB8…51DE	CE03975605E1…982F	CE03975605E1…982F	✓
1273	EscrowFinish	tesSUCCESS	5520CCDA3A06…2AEB	21A8567E0254…218C	21A8567E0254…218C	✓
1275	EscrowCreate	tesSUCCESS	CAFEB29DE305…9B40	8A77CC3A91DA…FDD6	8A77CC3A91DA…FDD6	✓
1276	LoanSet	tesSUCCESS	FDFA84B5FB3D…4066	9F4F19EEFE24…2E04	9F4F19EEFE24…2E04	✓
1321	LoanManage	tesSUCCESS	5AED6C886261…12A2	C150D36BE854…6637	C150D36BE854…6637	✓

> Full 64-character values are in the JSON. An earlier network run (ledgers 57–76) had the PC as the only validator; it is kept as proof_net_1val_* and is not a two-validator result.

Own network · this morning's run (the live 'show me' console)

Re-run on the two-validator network on Sun morning after the laptop node restarted; these are the hashes the Q&A console queries live (evidence/proof_am_result.json, evidence/proof_am_crosscheck.json).

Ledger	Type	Result	Tx hash (full)	Both nodes
10475	Payment	tesSUCCESS	B07C706653016F46DA3878295D4F20138CA4B9CE150C852871CC6F1A3EA2B40D	same ledger hash ✓
10477	Payment	tesSUCCESS	D7F91B7FA08CE5310E5B2D339A77E701CB8E6C42FF1C9AF66BB4583D465752AA	same ledger hash ✓
10478	Payment	tesSUCCESS	E4298BFB34B0B6EB8744CE36ED4CA194303873D85D867F7F4DDE3FDD02BE029C	same ledger hash ✓
10479	MPTokenIssuanceCreate	tesSUCCESS	DB2272E9B98DBA6A1304D3E0EFA88047053CA02756150B822B23AEC439960492	same ledger hash ✓
10480	MPTokenAuthorize	tesSUCCESS	132E5008CCC6A74CC64B270178551CF8E18E8709139325EFB1A40976F813EAA5	same ledger hash ✓
10481	MPTokenAuthorize	tesSUCCESS	E72DDF49AF13EED611CCC4CF2D3C4466FC7E3022921F1B2A7868F24CAF26D196	same ledger hash ✓
10482	Payment	tesSUCCESS	32403AD722146C6AC7FE807D8C5A5C8A59D4B26862F0A364D87C8C197CE08F74	same ledger hash ✓
10483	VaultCreate	tesSUCCESS	1CC55C525A95421562AD33EAF887B41F05AC1550976C62A85BDF6D18DB736B8D	same ledger hash ✓
10484	VaultDeposit	tesSUCCESS	8DCCB81737F412463120F5AD05E64050C45F26B3DDD4679929C4D41D7F49A820	same ledger hash ✓
10485	LoanBrokerSet	tesSUCCESS	40CAE49E9241B6DCE7FB2886E072DEE61B12A90793CBA2DD68BB8DF014E77530	same ledger hash ✓
10486	LoanBrokerCoverDeposit	tesSUCCESS	245097B42CA32DF06BBD7524D437F545CC5AC39E1F553B135BD1BFFE6D4C8ECE	same ledger hash ✓
10487	EscrowCreate	tesSUCCESS	D16A97DCD01E2CFF3E47BB276C2A329805EF55868F87AD786011AF8241EF262D	same ledger hash ✓
10488	LoanSet	tesSUCCESS	0C906B4D852CFADF2A2AF51FF59344F1F809FEB2C66226D6B9646543254D883F	same ledger hash ✓
10491	EscrowFinish	tecPLEDGED	DCCF16C158149B492D5313D13996A04E4BCAFDF8C05C5BEEB1661A5BB2797B8E	same ledger hash ✓
10492	EscrowFinish	tecPLEDGED	633DD9FBB6981EF8C87D71F6D432DB1EC3681429FB08F99122300EF801647380	same ledger hash ✓
10493	EscrowCancel	tecPLEDGED	E574C78A0F5E7DDB21DACA5421CED451E1A225DCE51E5EBF73C59700634BDA9B	same ledger hash ✓
10494	LoanPay	tesSUCCESS	1787B85B39307BE505443352ABE2E11266C359CD1B022C560DFA05225E7E5BA6	same ledger hash ✓
10495	LoanPay	tesSUCCESS	CE51B9C9B54B15B228C06DF0D5A1E4545E4DE1297A2BACEF6F44CEB75FAF7C21	same ledger hash ✓
10496	LoanPay	tesSUCCESS	64C588A2BD4A278F2C9D565C335D55F02A79A151118E476A58101CB714A75303	same ledger hash ✓
10497	EscrowFinish	tesSUCCESS	882275EE494C3AA78831C6FC872A17158E4FC67C3BC592D60560990724B64FA2	same ledger hash ✓
10498	EscrowCreate	tesSUCCESS	DE30F60D99ED67F55A9FAEDED78EE0787263507A396A918561F1B42288B78BB2	same ledger hash ✓
10499	LoanSet	tesSUCCESS	BBE154371C77FAB0FB8AA127CC99FDC9E3EFCE0D89167585CE3FFE5CBB516035	same ledger hash ✓
10544	LoanManage	tesSUCCESS	1DAF79EBDFE2471A5E1A072314011455D3406592E94066476A67D2961133F14C	same ledger hash ✓

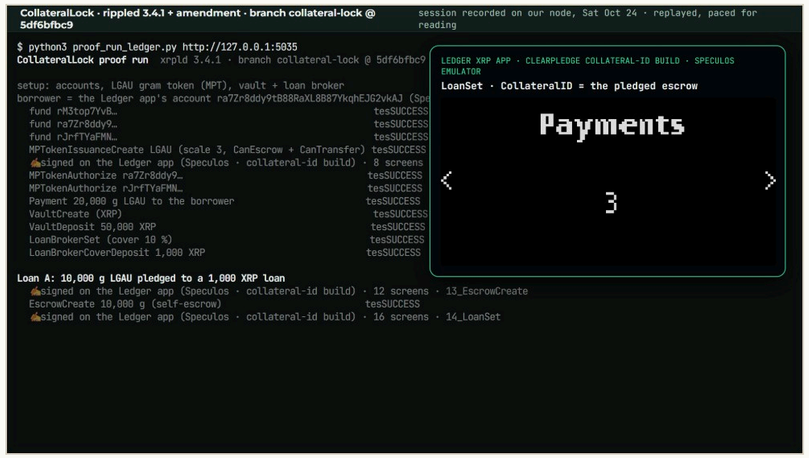
23 / 23 agree · build string 9ce853322 (same tree as 5df6bfbc9).

Own node · the recorded proof clip

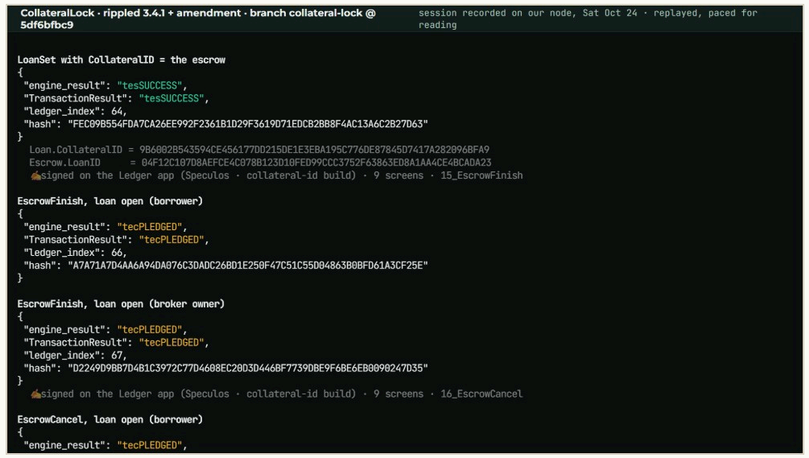
Frames from CollateralLock_proof.mp4, the run-3 recording used in the submission video: refusal while the loan is open, release after payoff, the default sweep.

OWN NODE · standalone · hashes local

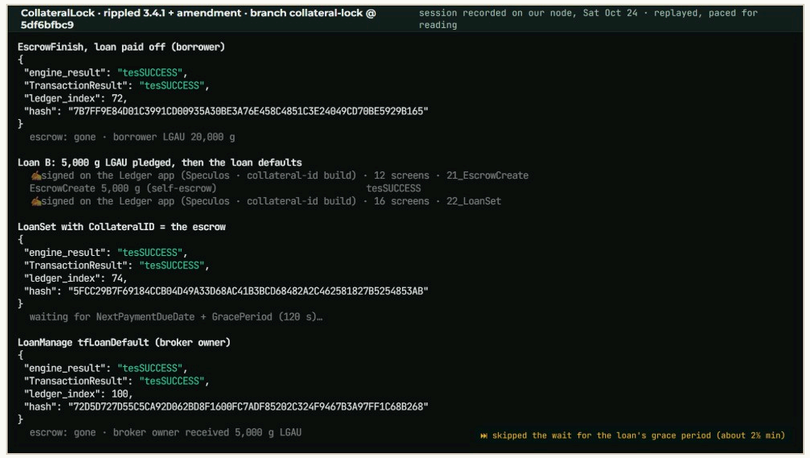
EMULATOR · Speculos



t = 12 s



t = 22 s



t = 33 s

The device overlay in these frames is the Ledger app in Speculos.

Check it yourself

Nothing here needs our machines except the own-node and own-network replays, and those come with full JSON.

SOURCE · GIT

PUBLIC DEVNET · anyone can check

OWN NODE · standalone · hashes local

1 · Rebuild the code

```
git clone https://github.com/XRPLF/rippled.git && cd rippled
git checkout -b collateral-lock 3.4.1      # 00e640751460c137…
git am /path/to/patches/*.patch
git rev-parse 'HEAD^{tree}'                   # expect fe497140150f7f4c2455a36594d8d2c7d9db0896
sha256sum /path/to/patches/*.patch          # compare with page 3
```

2 · Run the tests

```
mkdir build && cd build                     # conan 2 with a default profile (conan profile detect)
conan install .. --output-folder . --build missing -s build_type=Release -o "&:tests=True"
cmake -DCMAKE_TOOLCHAIN_FILE=build/generators/conan_toolchain.cmake -DCMAKE_BUILD_TYPE=Release -Dxrpld=ON
-Dtests=ON ..
cmake --build . --parallel
./xrpld --unittest=CollateralLock          # 7 cases, 687, 0 failures
```

3 · Open the public Devnet hashes

```
https://devnet.xrpl.org/transactions/3B32B17240A4BD0E8D6D75DA97D813040B6124C90151E8B06EBAF9AD14D79D26
https://devnet.xrpl.org/transactions/8E74D68A1EF3DC23DFF3FB32B827BD3855BDEF37252729E1813F20446876700B
https://devnet.xrpl.org/transactions/E100E8DDDE102306067F3BF20E6EC6058E7E348B5E4E2E4A8D91FE7D27358FEB
```

4 · Replay the own-node run

```
cd proof && touch validators.txt            # xrpld.cfg names validators.txt; standalone needs no
validators
/path/to/rippled/build/xrpld --conf xrpld.cfg -a --start &   # standalone, amendments enabled
python3 proof_run.py http://127.0.0.1:5035 my_run.json           # node-RPC signing
# or with the Ledger app in Speculos: proof_run_ledger.py + ledger_signer.mjs
```

5 · Check the recorded JSON

```
evidence/proof_run_result.json      run 3 (each submitted tx_blob kept, so signatures re-check)
evidence/proof_2val_result.json     two-validator run
evidence/proof_2val_crosscheck.json both nodes' validated ledger hash per tx
evidence/proof_am_result.json       this morning's two-validator run
evidence/proof_am_crosscheck.json   both nodes' ledger hash per tx (morning)
proof/device_screens/14_LoanSet/    the Ledger review screens + review.json
```

What we say plainly

- Own-node and own-network hashes can't be looked up on a public explorer; the JSON carries every result and ledger hash.
- The Ledger signing is in the Speculos emulator; the physical-device run is pending.
- CollateralLock is not on any public network; it needs review, a rebase onto develop and an amendment vote.
- No keys or seeds are included; the proof scripts derive rippled's public standalone genesis account at run time.